CNN GAN 자연어처리 이해하기

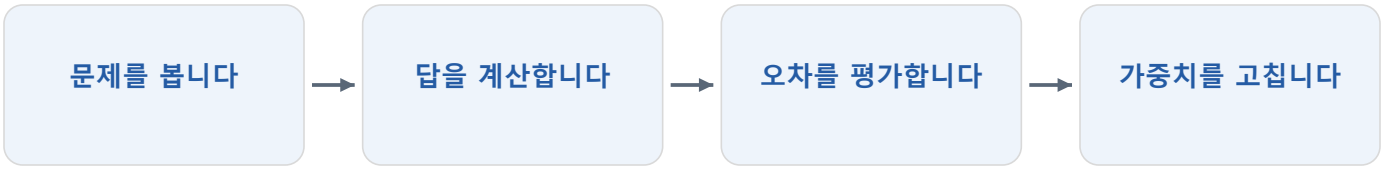
중학생을 위한 그림과 예시 중심 학습자료

사진에서 숫자를 읽고, 새로운 그림을 만들고, 문장을 번역하는 AI는 어떻게 작동할까요? 이 문서는 **무엇을 하는지와 왜 그런 방법을 쓰는지**를 함께 설명합니다. 복잡한 수식이나 코딩 지식은 필요하지 않습니다.

개념	무엇을 가리키나요	대표적인 질문
CNN	신경망의 구조	이 사진에 어떤 특징이 있나요?
GAN	두 신경망을 경쟁시키는 생성 모델의 틀	실제 데이터처럼 새로 만들 수 있나요?
자연어처리 NLP	사람의 언어를 다루는 분야	이 문장은 무슨 뜻이고 어떻게 답할까요?

먼저 AI의 학습부터 알아봅니다

신경망은 입력을 받아 결과를 계산하는 장치입니다. 안에는 결과에 영향을 주는 조절값인 **가중치**가 많이 있습니다. 학습은 평가 결과를 바탕으로 가중치를 조금씩 바꾸는 과정입니다.



예시 손글씨 7을 보고 1이라고 답했다면, 정답 7과 비교해 오차를 계산합니다. 그 오차를 줄이는 방향으로 가중치를 조정하고 여러 손글씨로 반복합니다.

왜 반복하나요? 사람마다 글씨가 다르기 때문입니다. 여러 모양에서 공통적인 특징을 배우면 처음 보는 글씨에도 답할 가능성이 높아집니다. 다만 학습을 많이 한다고 항상 잘 맞히는 것은 아닙니다.

학습과 사용의 차이 학습할 때는 가중치를 고칩니다. 보통 학습이 끝난 모델을 사용할 때는 저장된 가중치로 결과를 계산하며, 질문할 때마다 자동으로 다시 학습하지는 않습니다.

1 CNN은 작은 패턴을 찾아봅니다

CNN은 Convolutional Neural Network의 줄임말이며 합성곱 신경망이라고 부릅니다

모눈종이에 숫자 7을 그렸다고 생각해 보세요. 컴퓨터에는 7이라는 뜻 대신 칸마다 밝기를 나타내는 숫자가 들어갑니다. 이 칸 하나가 **픽셀**입니다.

작은 숫자판인 필터를 움직입니다

필터는 사진의 작은 부분과 계산하는 숫자판입니다. 아래 필터는 밝은 부분이 왼쪽에 있고 어두운 부분이 오른쪽에 있는 경계에 반응하는 간단한 예입니다.



같은 위치의 숫자끼리 곱해서 더합니다. 한 줄에서는 1×1 + 1×0 + 0×(-1) = 1입니다. 같은 줄이 세 개이므로 전체 반응값은 **3**입니다. 모든 칸이 1인 평평한 부분이라면 합은 0이 됩니다.

이 계산을 **합성곱**이라고 부릅니다. 필터를 옆과 아래로 움직여 반응값을 모으면, 어느 위치에 해당 패턴이 있는지를 보여 주는 **특징 지도**가 됩니다.

왜 작은 부분을 보고 같은 필터를 재사용하나요

작은 부분을 보는 이유 선이나 경계는 이웃한 픽셀 사이에서 드러납니다. 처음부터 모든 픽셀을 각각 연결하는 방식보다 조절값을 줄이면서 이런 관계를 살필 수 있습니다.

같은 필터를 쓰는 이유 세로선은 사진 왼쪽에도 오른쪽에도 있을 수 있습니다. 같은 탐지 방법을 여러 위치에 적용하면 위치마다 새 필터를 따로 배울 필요가 줄어듭니다.

근거 [1] 필터 숫자는 계산을 설명하기 위한 예시입니다. 실제 CNN에서는 필터 값도 학습합니다.

2 CNN은 특징을 조합하며 배웁니다

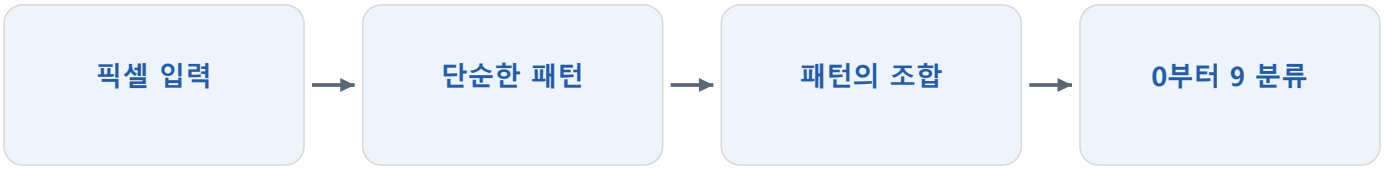
실제 학습 예시 손글씨 숫자 데이터 MNIST

MNIST는 0부터 9까지의 손글씨를 모은 실제 데이터입니다. 각 이미지는 가로 28칸, 세로 28칸입니다. 학습용 60,000장과 시험용 10,000장이 구분되어 있습니다. [2]



위 그림은 서로 다른 손글씨를 보여 주는 직접 그린 도식입니다. 실제 MNIST 이미지나 실행 결과는 아닙니다.

숫자 7을 알아보는 흐름



앞쪽 층은 선이나 경계 같은 단순한 특징을, 뒤쪽 층은 앞에서 얻은 특징의 더 복잡한 조합을 표현할 수 있습니다. 마지막 분류 부분이 그 특징들을 이용해 숫자를 고릅니다.

왜 여러 층이 필요한가요? 선 하나만으로는 1과 7을 구별하기 어렵습니다. 선의 방향과 배치 등을 함께 보면 더 복잡한 모양을 구별하는 데 도움이 됩니다. 각 층이 꼭 사람이 이름 붙일 수 있는 특징을 배우는 것은 아닙니다.

계산을 돕는 장치도 함께 씁니다

ReLU는 음수를 0으로 바꾸고 양수는 유지하는 계산입니다. 숫자를 2배한 뒤 3배해도 결국 6배하는 계산 하나로 합칠 수 있습니다. 층을 쌓을 때도 이런 단순한 합성에 머물지 않도록, ReLU 같은 꺾임이 있는 계산을 사이에 넣어 복잡한 관계를 표현합니다.

풀링은 작은 구역을 대표값으로 요약하는 방법입니다. 예를 들어 최대 풀링은 2, 5, 1, 3 중 가장 큰 5를 남깁니다. 계산량을 줄일 수 있지만 위치 정보 일부가 사라집니다. 모든 CNN에 꼭 들어가지는 않습니다.

왜 학습용과 시험용을 나누나요? 풀었던 문제만 다시 맞히면 실력을 확인하기 어렵습니다. 가중치를 고칠 때 쓰지 않은 시험용 글씨로 처음 보는 모양에도 잘 답하는지 확인합니다.

근거 [1] [2] [3] 실제 활용 예로 손글씨 인식과 사진 분류가 있습니다. 위 흐름은 이해를 위한 단순화입니다.

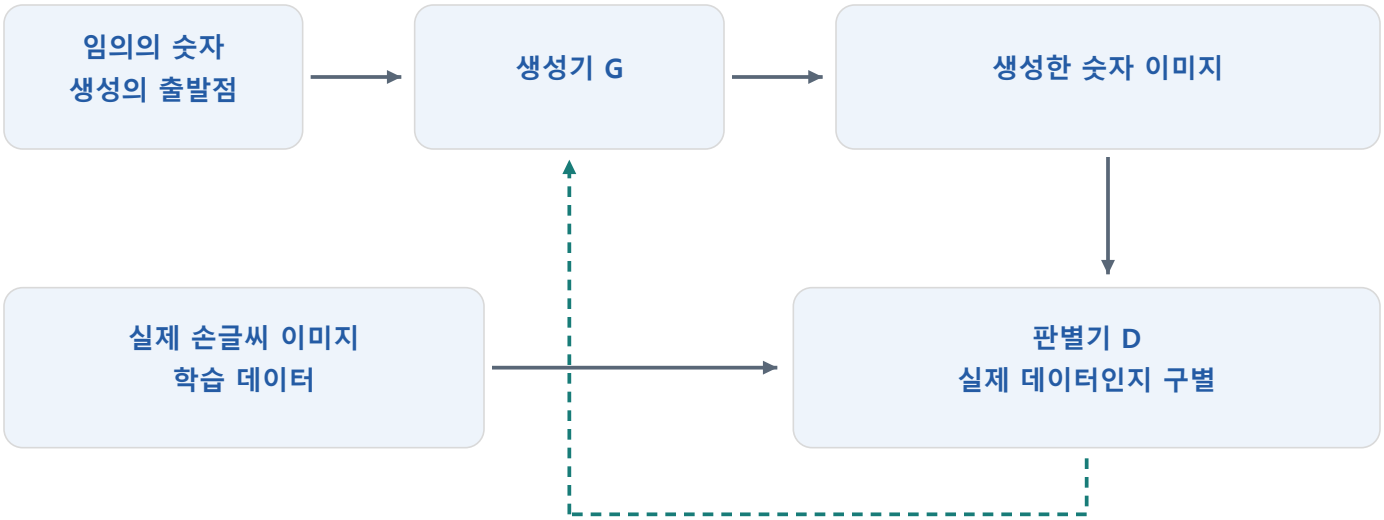
3 GAN은 만들기와 구별하기를 배웁니다

GAN은 Generative Adversarial Network의 줄임말이며 생성적 적대 신경망이라고 부릅니다

이제 숫자를 읽는 대신 **새로운 손글씨 숫자 이미지**를 만들고 싶다고 가정하겠습니다. 어느 픽셀을 얼마나 밝게 해야 자연스러운지, 사람이 모든 규칙을 적기는 어렵습니다.

생성기와 판별기가 서로 다른 일을 합니다

생성기 G는 임의의 숫자 묶음을 받아 이미지를 만듭니다. **판별기 D**는 실제 학습 이미지와 생성된 이미지를 구별하도록 배웁니다. 생성기는 판별기가 자신의 결과를 실제 데이터로 판단하도록 배웁니다.



평가에서 얻은 학습 신호로 생성기를 조정합니다

왜 판별기가 필요한가요

"숫자처럼 자연스럽다"를 판정하는 복잡한 규칙을 사람이 전부 정하는 대신, 판별기가 실제 이미지와 비교하며 평가 기준을 배웁니다. 생성기는 그 기준에 따라 자신의 출력을 개선할 수 있습니다.

두 신경망의 경쟁이란 서로 다른 목표를 가진 계산이라는 뜻입니다. 판별기는 더 잘 구별하려 하고, 생성기는 더 구별하기 어렵게 만들도록 가중치를 조정합니다. 사람이 말로 "잘 그렸다"고 평가하는 과정은 아닙니다.

근거 [4] [5] 기본적인 GAN의 학습 구조를 설명합니다.

4 GAN이 학습하는 순서를 살펴봅니다

실제 공개 예시 TensorFlow의 DCGAN 손글씨 생성 실습

TensorFlow의 공식 DCGAN 실습은 실제 MNIST 손글씨로 판별기를 훈련하고, 생성기가 손글씨 이미지를 만들도록 학습합니다. DCGAN은 합성곱 계열의 구조를 활용하는 GAN 예입니다. [5]

순서	어떤 자료를 사용하나요	무엇을 바꾸나요
1 생성	임의의 숫자를 생성기에 넣습니다.	현재 생성기로 이미지 한 묶음을 만듭니다.
2 판별기 학습	실제 이미지와 생성 이미지를 비교합니다.	두 종류를 더 잘 구별하도록 판별기 가중치를 바꿉니다.
3 생성기 학습	생성 이미지에 대한 판별기의 계산을 이용합니다.	실제라고 판단받도록 생성기 가중치를 바꿉니다.
4 반복	새로운 숫자와 이미지 묶음을 사용합니다.	두 학습 단계를 번갈아 반복합니다.

왜 임의의 숫자를 넣나요

같은 입력에 같은 계산만 하면 같은 결과가 나옵니다. 입력 숫자를 달리하면 서로 다른 결과를 만들 수 있는 출발점이 생깁니다. 생성기는 이 숫자 공간을 이미지로 바꾸는 법을 배웁니다. 다만 입력을 바꾼다고 다양한 결과가 저절로 보장되지는 않습니다.

생성기는 어떻게 고쳐야 할지 알 수 있나요

단순히 "가짜입니다"라는 판정만 받는 것이 아닙니다. 판별기의 계산을 거꾸로 따라가며, 생성기의 조절값을 어느 방향으로 조금 바꾸면 목표에 가까워질지 계산합니다. 이때 쓰는 방법을 **역전파**라고 합니다.

생성기를 고치는 단계에서는 판별기의 계산을 통과하지만, 보통 그 단계의 판별기 가중치는 고정합니다. 두 모델을 각각 어떤 목표로 고치는지 구분해야 합니다.

학습이 끝난 다음에는 어떻게 사용하나요

기본 GAN에서는 새 이미지를 만들 때 보통 **생성기만** 사용합니다. 판별기는 주로 학습을 돕는 역할입니다. 비슷한 결과만 만들거나 학습이 불안정해질 수 있으므로, 반복 횟수만 늘리면 해결된다고 볼 수는 없습니다.

근거 [4] [5] 기본 GAN의 학습 원리와 공개된 DCGAN 손글씨 생성 실습을 참고했습니다.

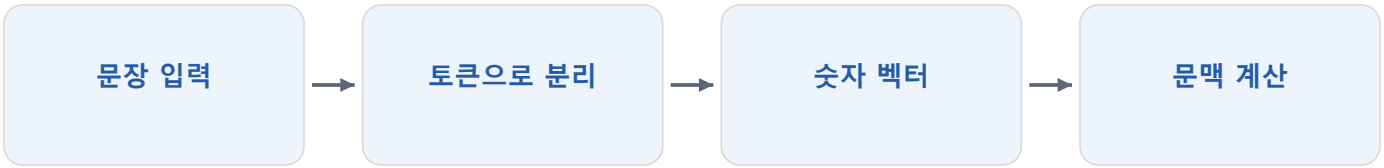
5 자연어처리는 언어를 다루는 분야입니다

NLP는 Natural Language Processing의 줄임말입니다

한국어와 영어처럼 사람이 사용하는 언어를 자연어라고 합니다. 자연어처리는 컴퓨터가 글이나 말의 내용을 분석하고 활용하도록 하는 분야입니다. 여기서는 **신경망으로 텍스트를 처리하는 방법**을 중심으로 살펴봅니다.

작업	입력 예시	출력 예시
감정 분류	이 영화는 정말 재미있었습니다.	긍정
번역	I like apples.	저는 사과를 좋아합니다.
정보 추출	체험학습은 금요일 9시에 출발합니다.	출발 시간은 금요일 9시
요약	준비물과 출발 시간이 담긴 긴 안내문	준비물과 시간을 짧게 정리한 문장

글을 토큰으로 나누고 숫자로 바꿉니다



토큰은 모델이 처리하는 텍스트 조각입니다. 단어 전체, 단어 일부, 글자나 문장부호 등이 될 수 있습니다. 예를 들어 설명용으로 "저는 / 사과를 / 좋아합니다 / ."처럼 나눌 수 있지만, 실제 분리는 모델에 따라 달라집니다.

왜 나누나요? 긴 문장을 한 덩어리로 외우는 대신 작은 단위를 조합해 처리할 수 있기 때문입니다. 단어 일부를 단위로 쓰면 새롭거나 드문 단어도 여러 조각으로 표현할 수 있습니다.

왜 숫자로 바꾸나요? 신경망은 숫자를 곱하고 더하는 장치이기 때문입니다. 글자를 계산에 쓸 수 있는 표현으로 바꿔야 합니다. 토큰에 붙인 번호 자체는 뜻이 아니며, 그 번호로 **임베딩**이라는 숫자 벡터를 찾아 사용합니다.

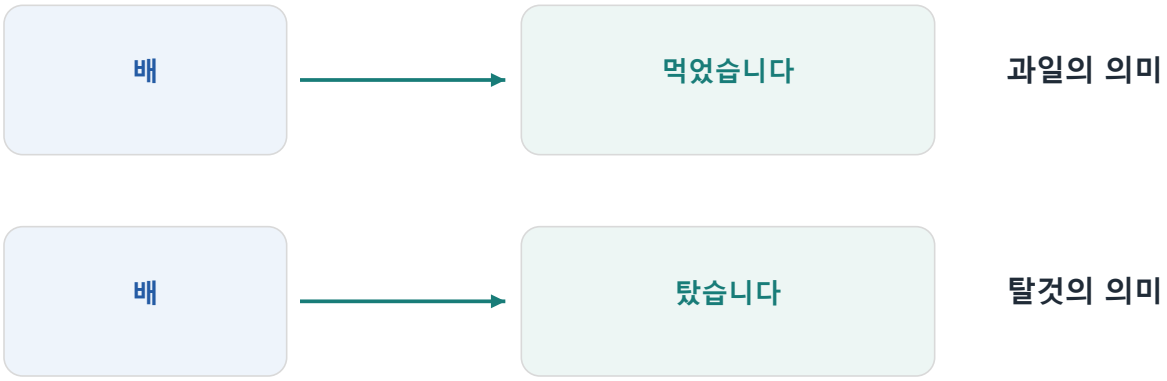
벡터란 무엇인가요? 여러 숫자를 순서대로 묶은 것입니다. 예를 들어 [0.2, -0.4, 0.7] 같은 형태입니다. 실제 모델의 숫자는 학습되며, 각 칸이 "음식 여부"처럼 사람이 정한 뜻을 꼭 가지지는 않습니다.

근거 [6] [7] 토큰 분리와 숫자 벡터는 설명을 위한 예시입니다.

6 문맥을 함께 봐야 뜻을 구별할 수 있습니다

같은 글자도 주변 표현과 순서에 따라 다르게 해석됩니다

"배를 먹었습니다"와 "배를 탔습니다"에서 배는 같은 글자입니다. 글자만 보면 과일인지 탈것인지 정할 수 없습니다. 주변 표현을 함께 살펴야 합니다.



Attention은 관계의 중요도를 계산합니다

Attention은 각 토큰을 처리할 때 다른 토큰의 정보를 얼마나 반영할지 계산하는 방법입니다. 위 문장에서 배를 해석할 때는 먹었습니다 또는 탔습니다라는 표현이 중요한 단서가 될 수 있습니다.

왜 필요한가요? 모든 토큰을 따로 처리하거나 똑같이 섞으면 문맥의 차이를 살리기 어렵습니다. 관계에 따라 정보를 다르게 반영하면 떨어진 표현도 연결해 처리할 수 있습니다. 그림의 화살표는 관계를 설명한 것으로, 실제 모델에서 측정한 Attention 값은 아닙니다.

Transformer는 Attention을 핵심 요소로 사용하는 신경망 구조입니다. 토큰의 순서 정보도 함께 활용합니다. "개가 사람을 물었습니다"와 "사람이 개를 물었습니다"를 구별하려면 표현의 배치와 관계를 살펴야 하기 때문입니다.

문장을 생성할 때는 다음 토큰을 이어갑니다

자기회귀 언어 모델은 "저는 아침에" 같은 앞부분을 보고 다음 토큰 후보의 확률을 계산합니다. 하나를 선택해 붙이고, 늘어난 문맥에서 다음 토큰을 다시 계산합니다. 후보 중 무조건 가장 확률이 큰 것만 고르는 방식만 있는 것은 아닙니다.

왜 다음 토큰을 예측하며 학습하나요? 학습 문장에 실제로 이어지는 토큰이 있으므로, 사람이 문장마다 별도 정답을 작성하지 않아도 예측과 비교할 수 있습니다. 그 과정에서 언어의 패턴을 배웁니다. 자연스러운 문장이 항상 사실인 것은 아닙니다.

근거 [7] [8] 자연어처리에는 분류처럼 새 문장을 만들지 않는 작업도 있습니다.

7 하나의 예시로 세 개념을 연결합니다

가상 프로젝트 손글씨 학습을 도와주는 서비스

학생이 쓴 숫자 사진을 읽고, 추가 연습용 이미지를 만들고, 사용 안내를 문장으로 제공하는 서비스를 설계한다고 가정하겠습니다. 아래는 가능한 설계 예이며 특정 제품의 내부 구조를 설명하는 것은 아닙니다.

기능	관련 개념	그 방법을 쓰는 이유
손글씨 사진이 7인지 판단합니다.	CNN	선과 모양의 배치에서 분류에 도움이 되는 특징을 찾습니다.
다른 모양의 연습용 손글씨 이미지를 만듭니다.	GAN	실제 손글씨 데이터와 닮은 이미지가 나오도록 두 신경망을 훈련할 수 있습니다.
사용 안내를 요약하거나 질문에 답합니다.	자연어처리	사용자의 문장과 안내문 속 정보를 처리해야 합니다.

CNN과 GAN은 함께 사용할 수 있습니다

CNN은 신경망 구조이고, GAN은 생성기와 판별기를 함께 훈련하는 틀입니다. 따라서 **GAN의 판별기에 CNN을 사용할 수 있습니다.** 이미지의 특징을 찾는 능력이 실제 이미지와 생성 이미지를 구별하는 데 도움이 될 수 있기 때문입니다. DCGAN이 관련 예입니다. [5]

자연어처리는 CNN이나 GAN과 같은 종류의 이름이 아닙니다

자연어처리는 해결하려는 작업이 속한 분야입니다. 그 작업을 수행하는 도구로 Transformer 같은 구조를 사용할 수 있습니다. CNN도 텍스트 처리에 사용할 수 있으므로 "CNN은 사진만 다룹니다"라고 외우면 범위를 너무 좁게 이해하게 됩니다.

헷갈리기 쉬운 부분을 확인합니다

① **모든 생성 AI가 GAN인 것은 아닙니다.** 생성하는 모델에는 여러 방식이 있습니다.
② **숫자 벡터 자체가 뜻을 완벽히 이해한다는 증거는 아닙니다.** 표현이 작업에 도움이 되도록 학습되는 것입니다.
③ **정확도와 자연스러움은 다릅니다.** 그럴듯한 이미지나 문장도 틀리거나 부정확할 수 있습니다.

직접 설명해 보세요. "CNN은 사진 특징을 뽑는 구조, GAN은 경쟁하며 생성 능력을 배우는 틀, 자연어처리는 언어를 다루는 분야입니다." 여기에 예시를 하나씩 붙여 보세요.

8 이해했는지 직접 확인해 보세요

정답을 보기 전에 한두 문장으로 이유까지 설명해 보세요

1 CNN 필터를 재사용하는 이유

사진 왼쪽의 세로선과 오른쪽의 세로선을 찾을 때, 왜 같은 필터를 여러 위치에 적용할까요?

---

2 작은 합성곱 계산

2쪽의 필터를 사용합니다. 사진의 작은 부분이 세 줄 모두 [0, 0, 1]이라면, 전체 반응값은 얼마일까요? 계산 이유도 적어 보세요.

---

3 GAN 판별기의 역할

생성기만 만들고 끝내지 않고, 실제 이미지와 생성 이미지를 구별하는 판별기를 왜 함께 훈련할까요?

---

4 입력을 숫자로 바꾸는 이유

자연어처리에서 토큰에 번호를 붙이는 일과 임베딩 벡터를 사용하는 일은 어떻게 다를까요?

---

5 문맥을 살펴보는 이유

"배가 달콤합니다"와 "배가 항구에 도착했습니다"에서, 배의 뜻을 구별하는 데 어떤 표현이 도움이 될까요?

---

6 개념 사이의 관계

"CNN을 판별기로 사용하는 GAN"이라는 말은 가능한가요? 그렇다면 두 용어가 각각 무엇을 가리키는지 설명해 보세요.

답을 못 쓰셨다면 해당 쪽의 "왜" 설명을 다시 읽어 보세요. 1번과 2번은 2쪽, 3번은 4~5쪽, 4번은 6쪽, 5번은 7쪽, 6번은 8쪽과 연결됩니다.

9 정답과 참고 자료

답의 표현이 달라도 같은 원리를 설명했다면 이해하신 것입니다

1번 같은 패턴이 여러 위치에 나타날 수 있기 때문입니다. 위치마다 별도 탐지 방법을 배우지 않고 가중치를 공유할 수 있습니다.

2번 한 줄의 값은 0×1 + 0×0 + 1×(-1) = -1입니다. 세 줄을 더하면 -3입니다. 음수도 경계 방향에 관한 반응이며, 실제 후속 계산에 따라 처리 방식이 달라집니다.

3번 자연스러운 이미지를 판정하는 규칙을 사람이 전부 적기 어렵기 때문입니다. 판별기가 실제 데이터와 비교해 배운 평가에서 생성기를 개선할 학습 신호를 얻습니다.

4번 토큰 번호는 어떤 조각인지 구별하는 식별자입니다. 임베딩은 그 토큰을 계산에 쓰기 위한 여러 숫자의 표현입니다. 번호의 크기 자체가 의미의 크기를 뜻하지 않습니다.

5번 달콤하다는 표현은 과일, 항구에 도착했다는 표현은 탈것이라는 해석에 도움이 됩니다. 모델은 이런 문맥 속 관계를 학습해 활용할 수 있습니다.

6번 가능합니다. CNN은 특징을 추출하는 신경망 구조이고, GAN은 생성기와 판별기를 훈련하는 틀이므로 판별기를 CNN으로 구성할 수 있습니다.

더 알아볼 때 참고할 원문

[1] Stanford CS231n Convolutional Neural Networks

[2] TensorFlow Datasets MNIST

[3] PyTorch Training a Classifier

[4] Goodfellow 외 Generative Adversarial Networks 2014

[5] TensorFlow Deep Convolutional GAN

[6] Google Machine Learning Crash Course Embeddings

[7] Google Large Language Models and Transformers

[8] Vaswani 외 Attention Is All You Need 2017

자료 확인일 2026년 10월 15일 | 그림과 계산 예시는 이해를 돕기 위해 직접 구성했습니다.